04

Fine-tuning: o que ele resolve — e o que não resolve

"Se eu treinar o modelo com os dados da minha empresa, ele para de errar sobre o meu projeto?" Essa é uma das perguntas mais caras do mercado — cara no sentido literal: quem responde errado gasta muito dinheiro para não resolver o problema que tinha. Este material explica o que o fine-tuning é, quando vale o investimento e por que ele quase nunca é a resposta para o problema que você provavelmente tem.

A IDEIA CENTRAL DESTE MATERIAL

Fine-tuning ensina comportamento, não fatos do seu projeto. Ele muda *como* o modelo responde — não *o que* ele sabe sobre o seu sistema. Para conhecimento específico e mutável, a resposta é contexto, não treinamento.

1 O que é fine-tuning

Fine-tuning é um treinamento adicional aplicado sobre um modelo base pronto. Em vez de treinar do zero — o que exige volumes absurdos de dados, tempo e infraestrutura — você parte de um modelo que já sabe um pouco de tudo e **refina o comportamento dele** com exemplos do seu domínio.

Tecnicamente, é a continuação do mesmo processo do treinamento original: o modelo vê seus exemplos, compara o que produziu com o que você esperava e ajusta os parâmetros na direção do acerto. A diferença está na escala e na delicadeza — o corpus é muito menor, muito mais específico, e o ajuste usa um learning rate baixo de propósito: a ideia é **refinar sem destruir** o que o modelo já aprendeu.

```
Treinamento base:
bilhões de exemplos genéricos → modelo que sabe um pouco de tudo

Fine-tuning:
centenas/milhares de exemplos específicos → modelo que mantém o
conhecimento geral e incorpora o comportamento do seu domínio
```

2 O que o fine-tuning melhora de verdade

Fine-tuning brilha em comportamentos que precisam ser consistentes em **todas** as interações, independente do prompt:

ALVO	EXEMPLO PRÁTICO
Tom e estilo	Comentários sempre em inglês, documentação em JSDoc, variáveis em camelCase — sem precisar pedir toda vez.
Convenções de código	Padrões de nomenclatura, estrutura de arquivos, estilo de tratamento de erros que se repetem em todo projeto.
Formato de output	JSON sempre na mesma estrutura, testes sempre acompanhando o código, perguntas de esclarecimento antes de implementar.
Vocabulário de domínio	Terminologia de saúde, jurídico, financeiro, industrial — onde o modelo genérico tropeça em nomenclatura.

3 O que ele NÃO melhora — a confusão mais cara

Aqui está o ponto que separa quem investe bem de quem queima orçamento. Fine-tuning **não injeta conhecimento dinâmico sobre o seu projeto**:

✓ FINE-TUNING RESOLVE

Como o modelo se comporta.

Em que estilo ele escreve.

Que padrões segue por default.

Como formata as respostas.

✗ FINE-TUNING NÃO RESOLVE

O que está na SPEC do projeto atual.

As decisões de arquitetura desta semana.

O que foi implementado ontem, o bug resolvido hoje.

Qualquer conhecimento que muda com o tempo.

Um modelo fine-tunado **continua alucinando** sobre o contexto do seu projeto — porque esse contexto não estava nos dados de treinamento e muda constantemente. Lembre do material 01: o modelo não sabe o que não sabe. Fine-tuning não cria essa consciência; apenas muda o sotaque com que ele inventa.

> Fine-tuning e RAG **não são concorrentes — são complementares**. Fine-tuning define o comportamento padrão. RAG (material 14) injeta o conhecimento específico no momento certo.

4 Como são os dados: pares de treinamento

O fine-tuning aprende com pares entrada/saída — centenas de demonstrações de como você quer que o modelo se comporte:

```
{
  "messages": [
    { "role": "user",
      "content": "crie um endpoint de criação de usuário" },
    { "role": "assistant",
      "content": "@router.post('/users', response_model=UserResponse)
                  async def create_user(...)" }
  ]
}
```

Cada par ensina um comportamento. E vale a regra de ouro dos dados: **a qualidade dos pares determina a qualidade do resultado**. O modelo aprende os erros dos seus exemplos com a mesma confiança com que aprende os acertos.

- **Mínimo para ver diferença:** a recomendação usual dos provedores fica entre 50 e 100 pares.
- **Para comportamento robusto:** 500 a 1.000 pares bem curados.
- **Curadoria é obrigatória:** cada par revisado por um especialista do domínio. Conversas reais contêm tentativas, erros e decisões revertidas — usá-las sem filtro ensina o modelo a errar.

5 Uma software house consegue fazer o próprio fine-tuning?

Tecnicamente, sim. Na prática, a resposta depende de três frentes:

Dados

É preciso transformar conversas reais de desenvolvimento que deram certo em pares de treinamento. Esse processo é manual, demorado e exige julgamento técnico para separar o bom exemplo do ruim. É o custo mais subestimado do projeto.

Infraestrutura — dois caminhos

CAMINHO	COMO FUNCIONA	PARA QUEM
API do provedor	Você envia os dados, paga por token de treinamento e recebe um modelo customizado hospedado pelo provedor.	A opção acessível — sem infraestrutura própria.
Infra própria	GPUs, frameworks como Hugging Face PEFT/LoRA, hospedagem do modelo resultante (Llama, Mistral etc.).	Volumes muito grandes ou exigência de modelo open source.

Manutenção — o custo escondido

Modelo fine-tunado **envelhece**. Quando as convenções mudam, quando a stack evolui, ele continua com o comportamento antigo — e retreinar significa coletar de novo, curar de novo, validar de novo. É um produto de software com ciclo de vida próprio, não um investimento único.

6 Quando faz sentido — e quando não

FAZ SENTIDO QUANDO…	POR QUÊ
Vertical muito especializada (saúde, jurídico, financeiro, industrial)	Vocabulário e padrões que não existem em volume no treinamento base. O ganho em precisão de terminologia é real.
Volume de uso muito alto (centenas de devs, dezenas de projetos)	O custo se amortiza contra os tokens gastos repetindo as mesmas instruções em todo prompt.
Padrões estáveis e universais na empresa	Convenções que raramente mudam e valem para todos os clientes/ projetos.

Para a maioria das software houses, porém, o retorno não justifica — porque o problema real não é o comportamento padrão do modelo, e sim a **ausência de contexto específico por projeto**:

INVESTIR EM FINE-TUNING

Alto custo de curadoria + treinamento + manutenção contínua.

Melhora o comportamento padrão — **não resolve o contexto por projeto**.

Fica para trás a cada geração nova de modelos; retreinar custa.

INVESTIR EM ENGENHARIA DE CONTEXTO

Custo baixo, cresce junto com cada projeto.

Resolve exatamente o contexto específico.

Funciona com qualquer modelo — e melhora de graça quando sai um modelo novo.

7 Fine-tuning vs RAG vs Engenharia de contexto

As três técnicas resolvem problemas diferentes — e se combinam:

TÉCNICA	RESOLVE	TIPO DE CONHECIMENTO
Fine-tuning	Comportamento padrão: estilo, formato, vocabulário	Estático — muda raramente
RAG (material 14)	Conhecimento específico injetado na hora da consulta: a decisão de ontem, o bug desta semana	Dinâmico — muda o tempo todo
Engenharia de contexto (materiais 05–12)	A arquitetura que entrega o contexto certo, no formato certo, no momento certo: SPEC, PLAN, agents, hooks	Estruturado — cresce com o projeto

> Comportamento padrão correto **+** conhecimento dinâmico do projeto **+** contexto estruturado da sessão **=** um modelo que age como especialista do seu domínio conhecendo profundamente o seu projeto.

8 Erros mais comuns

ERRO	POR QUE DÓI
Fine-tuning para resolver problema de contexto	Se o modelo não segue as convenções do projeto, quase sempre falta contexto — um CLAUDE.md bem escrito resolve em 30 minutos o que o fine-tuning tentaria resolver com semanas e orçamento.
Dados sem curadoria	Conversas reais têm erros e decisões revertidas. O modelo aprende tudo — inclusive o que você não queria ensinar.
Poucos exemplos	Menos de 50 pares raramente produz diferença mensurável. O padrão precisa se repetir para ser internalizado.
Não versionar os dados de treinamento	Os pares são um ativo tão valioso quanto o código. Sem eles, você não reproduz o comportamento nem entende por que o modelo age como age.
Retreinar sem validação	Fine-tuning pode melhorar um comportamento e degradar outro silenciosamente. Valide em casos reais antes de colocar em produção.

▸ **COLA PARA FIXAR**

- Fine-tuning é **treinamento adicional** sobre um modelo pronto: refina comportamento sem apagar o conhecimento geral.
- Ele melhora **como** o modelo responde (estilo, formato, vocabulário) — nunca **o que** ele sabe sobre o seu projeto.
- Modelo fine-tunado **continua alucinando** sobre contexto que não estava no treinamento.
- Qualidade dos pares > quantidade: curadoria por especialista é obrigatória.
- Faz sentido para **verticais especializadas, volume alto e padrões estáveis** — e envelhece, exigindo manutenção.
- Para o problema típico de software house, **engenharia de contexto** entrega mais, custa menos e melhora a cada modelo novo.